

Trabajo Final Integrador (TFI)

Food Store

Modelado, carga masiva, consultas, seguridad y concurrencia — MySQL 8.0

Alfredo Castillo — Etapas 1 y 4 (Modelado · Seguridad)

Cesia Castilla — Etapas 2 y 5 (Datos masivos · Concurrencia)

Rocío Agüero — Etapa 3 (Consultas · Rendimiento)

SQL puro y procedimientos almacenados · Sin código Java de conexión · Mismo dominio que el TPI de Programación 2

Objetivo del Proyecto

Qué cubre el TFI y cómo se organizó el trabajo en equipo

DOMINIO

Food Store

Mismo modelo de negocio que Programación 2: categorías, usuarios, productos, pedidos y detalles de pedido.

VOLUMEN

+16.000 registros

Carga masiva generada con SQL puro, superando el mínimo de 10.000 filas exigido por la consigna.

TECNOLOGÍA

MySQL 8.0

5 scripts por etapa + procedimientos almacenados, sin JDBC ni código de conexión Java.

5 etapas, 3 integrantes: Modelado (Alfredo) · Datos masivos (Cesia) · Consultas (Rocío) · Seguridad (Alfredo) · Concurrency (Cesia).

Modelado y Constraints

Del DER al esquema relacional — 01_esquema.sql

DER

- 5 entidades: `categorias`, `usuarios`, `productos`, `pedidos`, `detalles_pedidos`.
- Relaciones 1:N entre categoría → producto y usuario → pedido.
- `detalles_pedidos` como tabla intermedia entre pedido y producto, con atributos propios (cantidad, precio).

CONSTRAINTS

- **PK** autoincremental en las 5 tablas.
- **FK** con `ON DELETE CASCADE` solo entre pedido y su detalle.
- **UNIQUE** en `email` y nombres de categoría.
- **CHECK** en precio, stock, cantidad y enums (rol, estado, forma de pago).

Idempotencia: el script usa `DROP TABLE IF EXISTS` en orden inverso de dependencias, así puede reejecutarse sin conflictos.

Validación Práctica del Modelo

Probar la robustez del diseño antes de la carga masiva

Prueba	Resultado esperado
Inserción válida	Categoría/producto nuevo, respeta todos los CHECK
PK duplicada	Error 1062 — Duplicate entry
FK inexistente	Error 1452 — Cannot add or update a child row
CHECK violado (precio negativo)	Error 3819 — Check constraint violated

Por qué importa: validar el modelo físico antes de cargar miles de registros evita propagar datos inconsistentes en las etapas siguientes.

Generación de Datos Masivos

03_carga_masiva.sql — SQL puro, sin scripts externos

MECANISMO

- `WITH RECURSIVE` genera series numéricas 1..N sin tablas auxiliares persistentes.
- `ROW_NUMBER() OVER (ORDER BY id)` sobre las tablas semilla, combinado con el operador módulo (`%`) para distribuir las FKs siempre contra ids reales.
- Precio, stock y cantidad generados con `RAND()` acotado a los rangos permitidos por los `CHECK` del esquema.

Tabla	Filas nuevas
usuarios	1.000
productos	1.000
pedidos	4.000
detalles_pedidos	10.000

Garantía de Integridad

Cuadro de verificaciones sobre 16.000+ registros reales

FKS

0 huérfanas

Cada fila generada toma su FK de un JOIN contra una CTE de ids reales: nunca se inventa un id.

CARDINALIDAD

100% pedidos con detalle

La condición `(p.id % 4) + 1` garantiza entre 1 y 4 detalles por pedido, nunca cero.

TOTALES

Recalculados

`pedidos.total` se recalcula con `UPDATE ... JOIN` sumando los subtotales reales.

Verificado contra MySQL 8.0 real: 16.003 registros totales, 0 valores fuera de rango, ejecución completa en menos de 1 segundo.

Consultas Complejas y Vistas

05_consultas.sql · 06_vistas.sql

4 CONSULTAS

- **JOIN:** productos más vendidos por categoría.
- **JOIN:** pedidos por usuario con su detalle completo.
- **GROUP BY + HAVING:** clientes con facturación > \$1.000.
- **Subconsulta:** productos con stock por debajo del promedio.

VISTA

- `vista_pedidos_detallados` combina pedido, usuario, detalle y producto en una sola consulta reutilizable.
- Simplifica reportes y sirve de base para las mediciones de índice de la etapa.

Utilidad real: cada consulta responde una pregunta de negocio concreta (rotación de productos, fidelización de clientes, reposición de stock).

Índices y Medición de Rendimiento

05_explain.sql — EXPLAIN ANALYZE con y sin índice

Caso	Columna	Conclusión
Igualdad	<code>usuarios.email</code>	Mejora clara: columna de alta cardinalidad (única por usuario)
Rango	<code>productos.stock</code>	Mejora clara: <code>EXPLAIN</code> pasa de table scan a index range scan
Igualdad	<code>pedidos.estado</code>	Sin mejora significativa: baja cardinalidad (solo 4 valores posibles)

Hallazgo conectado con la teoría (Unidad 6): un índice sobre una columna con pocos valores únicos "no aporta" — por eso en `04_indices.sql` se reemplazó por un índice compuesto (`estado, created_at`) , útil para filtrar y ordenar a la vez.

Usuario y Vistas de Seguridad

07_seguridad.sql — Mínimo privilegio

USUARIO MÍNIMO

- `operador_foodstore` creado sin privilegios (`GRANT USAGE` inicial).
- Solo recibe `SELECT` sobre 2 vistas específicas, nunca sobre las tablas reales.

VISTAS PROTECTORAS

- `vw_usuarios_publico` : email enmascarado (`j***@gmail.com`) en vez de la dirección completa.
- `vw_productos_disponibles` : solo productos activos con stock > 0.

Principio de mínimo privilegio (Unidad 8): el usuario de aplicación nunca toca una tabla directamente, solo las vistas curadas para su rol.

Anti-Inyección y Pruebas de Integridad

sp_buscar_producto_seguro — sin SQL dinámico

PROCEDIMIENTO PARAMETRIZADO

- Recibe `p_nombre_buscar` como parámetro tipado, nunca concatena texto en la consulta.
- Prueba documentada: se envía `' OR '1'='1` como entrada maliciosa — el motor lo trata como texto literal, no como código SQL.
- Resultado: grilla vacía o coincidencia literal, sin filtración de datos. Ataque neutralizado.

Pruebas de integridad reutilizadas de Etapa 1: PK duplicada, FK inválida y CHECK violado, documentadas con su código de error real (1062 / 1452 / 3819).

Transacciones y Retry ante Deadlock

08_transacciones.sql — sp_confirmar_pedido

TRANSACCIÓN REAL

- Confirma un pedido: descuenta stock de cada detalle y cambia el estado, todo en una sola transacción.
- `SELECT ... FOR UPDATE` bloquea pedido y productos involucrados.

RETRY AUTOMÁTICO

- `DECLARE EXIT HANDLER FOR SQLSTATE '40001'` captura el deadlock específicamente.
- Hasta 2 reintentos con backoff creciente (0.3s, 0.6s); cada intento queda registrado en `log_transacciones`.

Probado en MySQL real: camino feliz → pedido `CONFIRMADO` ; camino de error (stock insuficiente) → `ROLLBACK` limpio, sin cambios parciales.

Concurrencia y Niveles de Aislamiento

09_concurrencia_guiada.sql — simulación con 2 sesiones

Prueba	Resultado real obtenido
Deadlock (locks cruzados)	ERROR 1213 (40001) : Deadlock found when trying to get lock
REPEATABLE READ	Segunda lectura no cambia (136 → 136) aunque la otra sesión ya confirmó
READ COMMITTED	Segunda lectura refleja el cambio de inmediato (58 → 8)

Conclusión: InnoDB detecta y resuelve el deadlock automáticamente; la elección de nivel de aislamiento depende de si el caso de uso necesita una foto estable (REPEATABLE READ) o el dato más reciente (READ COMMITTED).

Conclusiones

Lo que construimos como equipo en Bases de Datos I

INTEGRIDAD

PK · FK · CHECK

Validadas con pruebas reales en las 5 etapas, sobre un esquema idempotente.

RENDIMIENTO

+16.000 registros medidos

Índices justificados con EXPLAIN y conectados a la teoría de cardinalidad.

SEGURIDAD Y CONCURRENCIA

Mínimo privilegio · Retry

Anti-inyección documentada y transacciones con recuperación ante deadlock.

Equipo Food Store (BD-I): Alfredo Castillo (Etapas 1 y 4) · Cesia Castilla (Etapas 2 y 5) · Rocío Agüero (Etapa 3) — UTN, Bases de Datos I. ¡Gracias!